

Cohere:

The screenshot displays a REST client interface with a POST request to `{{baseUrl}}/v2/embed`. The request body is raw JSON, and the response is a 200 OK status with a 529 ms response time and 6.5 KB of data. The response body is shown in JSON format, containing an ID, the input text, and a list of float embeddings.

Request:

```
1 {
2   "model": "embed-v4.0",
3   "texts": [
4     "This is a test sentence."
5   ],
6   "input_type": "search_document",
7   "embedding_types": [
8     "float"
9   ]
}
```

Response:

```
1 {
2   "id": "7dec4664-2b68-4ac9-b1cf-fb23b4547677",
3   "texts": [
4     "This is a test sentence."
5   ],
6   "embeddings": {
7     "float": [
8       [
9         -0.00005964969,
10        0.0071988655,
11        0.030104347,
12        -0.0090531185,
13        -0.0046629016,
14        -0.004472023,
15        -0.02279641,
16        -0.015488468,
17        0.0038993855,
18        -0.0049083172,
19        -0.019415123,
20        -0.04079357,
```

Voyage:

The screenshot displays a REST client interface with a POST request to `{{base_url}} /v1/rerank`. The request body is raw JSON. The response is a 200 OK status with a JSON body containing a list of reranked documents.

Request:

```
1 {
2   "query": "What is RAG?",
3   "documents": [
4     "Retrieval Augmented Generation combines retrieval with a language model.",
5     "MongoDB is a NoSQL database.",
6     "RAG retrieves relevant documents before generating an answer.",
7     "The Eiffel Tower is located in Paris."
8   ],
9   "model": "rerank-2.5",
10 }
```

Response:

```
1 {
2   "object": "list",
3   "data": [
4     {
5       "relevance_score": 0.7421875,
6       "index": 2,
7       "document": "RAG retrieves relevant documents before generating an answer."
8     },
9     {
10      "relevance_score": 0.62890625,
11      "index": 0,
12      "document": "Retrieval Augmented Generation combines retrieval with a language model."
13     },
14     {
15      "relevance_score": 0.3359375,
16      "index": 3,
17      "document": "The Eiffel Tower is located in Paris."
18     }
19   ],
20   "model": "rerank-2.5",
21   "usage": {
22     "prompt_tokens": 12,
23     "completion_tokens": 0,
24     "total_tokens": 12
25   }
26 }
```

Jina:

The screenshot displays a REST client interface with a POST request to `{{baseUrl}} /classify`. The request body is a JSON object with the following structure:

```
1 {
2   "input": [
3     "I am unable to login to my account",
4     "How can I upgrade my subscription?",
5     "The application is running very slowly"
6   ],
7   "labels": [
8     "Login Issue",
9     "Billing",
10    "Performance"
11  ],
12  "model": "jina-embeddings-v2-base-en"
13 }
```

The response status is **200 OK** with a response time of 5.07 s and a size of 899 B. The response body is a JSON object:

```
2  "data": [
3    {
4      "object": "classification",
5      "index": 0,
6      "prediction": "Login Issue",
7      "score": 0.38742159508011575,
8      "predictions": [
9        {
10         "label": "Login Issue",
11         "score": 0.38742159508011575
12       },
13       {
14         "label": "Billing",
15         "score": 0.311441698829119446
16       },
17       {
18         "label": "Performance",
19         "score": 0.29999999999999996
20       }
21     ]
22   }
23 ]
```

Sarvam:

Sarvam AI APIs > Language Identification

POST

{{baseUrl}} /text-lid

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

1

{

2

"input": "வணக்கம்"

3

}

Body

Cookies

Headers 17

Test Results

200 OK

306 ms

756 B

Save Response

{}

JSON

Preview

Visualize

1

{

2

"request_id": "20260807_e165c520-2a25-42ba-a0d2-c28956eed58f",

3

"language_code": "ta-IN",

4

"script_code": "Tamil"

5

}

Sarvam AI APIs > Speech To Text

POST

{{baseUrl}} /speech-to-text

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

Key	Value	Description	Bulk Edit
file	audio (1).wav		

Key	Value	Description
-----	-------	-------------

Body

Cookies

Headers 17

Test Results

200 OK

519 ms

799 B

Save Response

{}

JSON

Preview

Visualize

1

{

2

"request_id": "20260807_ff43dd83-2bf4-4704-bbe9-9b6c7c7ad087",

3

"transcript": "Hello, Hi everyone.",

4

"language_code": "en-IN",

5

"language_probability": 0.672

6

}

The screenshot displays the RAG Pipeline web application. The left sidebar contains navigation links: 'Embeddings & Store' (Create and manage embeddings), 'Query Preprocessing' (Transform & expand queries), 'Vector Search' (Semantic vector search), 'BM25 Search' (Keyword-based search), 'Hybrid Search' (Combined BM25 + Vector), 'Score Fusion' (BM25+Vector fusion reranking), 'Summarize & Dedup' (AI summarization & deduplication), 'Prompt & Schema' (Configure prompt templates & JSON schemas), and 'Settings' (Configure environment). The central workspace is titled 'RAG Pipeline' and shows the 'Prompt & Schema' configuration page. It includes a 'Test Prompt Template' section with a text input area containing the prompt: 'Given the doctor wants to view only critical patients, when the "Emergency" critical priority should be visible.' Below this is a 'Sample Test Cases (JSON)' section with a JSON editor containing test cases for a laboratory module. The right-hand panel shows a network monitoring interface with a timeline and a list of network requests. The selected request is a POST to '/api/v1/pipeline/test' with a JSON payload containing test cases and a user story.

The screenshot displays the Swagger UI for a REST API. The top navigation bar shows the API name "RAG Backend API Collection-June" and the current endpoint "Preprocess Query". The "POST" method is selected, and the URL is "{{base_url}} /api/search/preprocess". The "Body" tab is active, showing the request body in raw format. The request body is a JSON object with the following structure:

```
{
  "query": "password reset OTP verification",
  "options": {
    "enableAbbreviations": true,
    "enableSynonyms": true,
    "maxSynonymVariations": 5,
    "customAbbreviations": {},
    "customSynonyms": {},
    "smartExpansion": false,
    "preserveTestCaseIds": true
  }
}
```

The response body is also shown in raw format, indicating a 200 OK status with a response time of 5 ms and a size of 1.21 KB. The response body is a JSON object with the following structure:

```
{
  "original": "password reset OTP verification",
  "normalized": "password reset otp verification",
  "abbreviationExpanded": "password reset one time password verification",
  "synonymExpanded": [
    "password reset one time password verification",
    "credential reset one time password verification",
    "passphrase reset one time password verification",
    "passcode reset one time password verification",
    "password change one time password verification"
  ],
  "metadata": {
    "testCaseIds": [],
    "abbreviationMappings": [
      {
        "abbreviation": "otp",
        "original": "one time password",
        "normalized": "one time password"
      }
    ]
  }
}
```

RAG Backend API Collection-June > Results Processing > Summarize Results

POST

{{base_url}} /api/search/summarize

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "results": [
3     {
4       "id": "TC_001",
5       "title": "Login functionality test",
6       "description": "Test user login with valid credentials",
7       "steps": "1. Open login page\n2. Enter username\n3. Enter password\n4. Click login",
8       "expectedResults": "User should be logged in successfully"
9     }
10  ],
11  "summaryType": "concise",
12  "query": "login test"
13 }
```

Body

Cookies

Headers 8

Test Results

200 OK

855 ms

1.44 KB

Save Response

JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "summary": "The search results for \"login test\" appear to be related to testing the login functionality of
4     a system. The top result, TC_001, is a test case titled \"Login functionality test\" that aims to verify
5     user login with valid credentials. This suggests that the search results are focused on ensuring that the
6     login process works correctly and securely.\n\nThe result implies that the test case is designed to
7     validate the login functionality, possibly as part of a larger quality assurance or testing protocol. The
8     fact that it specifies \"valid credentials\" indicates that the test is intended to confirm that
9     authorized users can successfully log in to the system.\n\nOverall, the search results seem to be
10    centered around testing and validating the login process, which is a critical aspect of system security
11    and functionality. If you're looking to test or troubleshoot login issues, these results may provide a
12    useful starting point.",
13   "summaryType": "concise",
14   "resultCount": 1,
15   "model": "llama-3.3-70b-versatile",
16   "tokens": {
17     "prompt": " ",
18     "completion": " "
19   }
20 }
```